# چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

**نسخهٔ ۲۰۲۶**

روز ۱۹

## امنیت — بخش ۱: مدل‌سازی تهدید، Keystore، مجوزها، Intentها و WebView



**اسماعیل مرادی**

@esmaeeil-moradi700

# چرا امنیت روز نوزدهم است؟

سؤال‌های امنیتی می‌سنجند که آیا می‌توانید اپ خودتان را **از چشم یک مهاجم** ببینید. از نسخهٔ پارسال این چالش، زمین بازی عوض شده است:

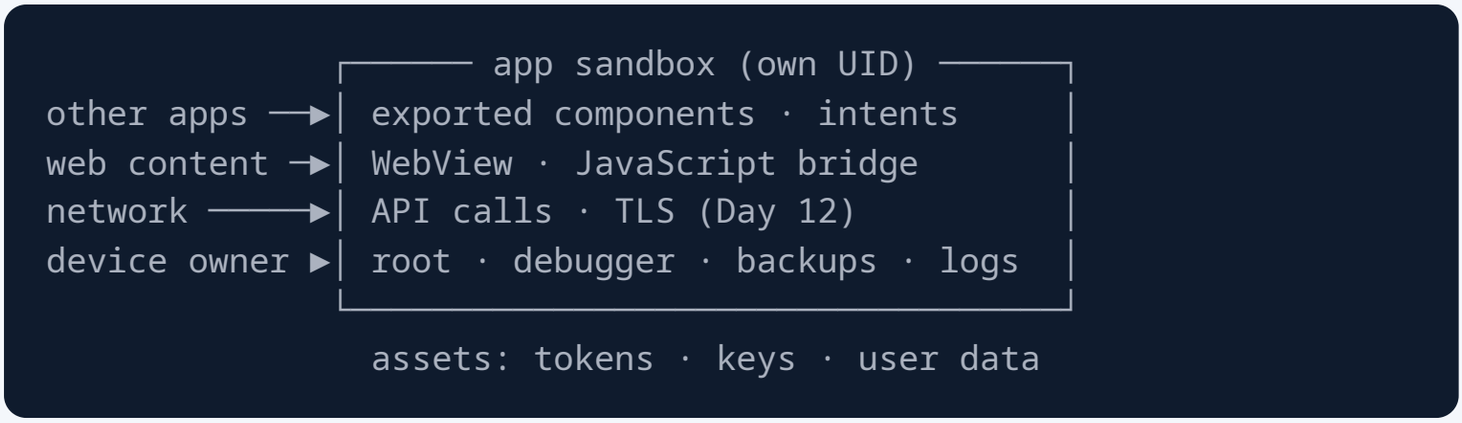
- کتابخانهٔ Jetpack Security **منسوخ شده** — همان که `EncryptedSharedPreferences` و `EncryptedFile` را داشت؛ حالا باید خود Keystore را بلد باشید
- هر نسخهٔ اندروید پیش‌فرض‌ها را سخت‌گیرانه‌تر کرده: `exported` صریح، `PendingIntent` تغییرناپذیر، دسترسی جزئی به عکس‌ها، محدودیت intentهای ضمنی
- گوگل‌پلی سیاست‌های مجوز و فرم Data safety را اعمال می‌کند — و SDKهای شما هم جزو آن حساب می‌شوند
- اپ‌ها حالا refresh token، پروندهٔ سلامت و پرداخت نگه می‌دارند — ریسک واقعی است

| سطح | سؤال‌ها |
| --- | --- |
| جونیور | مدل امنیتی پلتفرم، داده کجا ذخیره می‌شود، مجوزهای runtime |
| مید | دسترسی به رسانه و دسترسی جزئی، کامپوننت‌های exported و intentها، PendingIntent، WebView |
| سینیور | Android Keystore، ذخیره‌سازی بعد از security-crypto، نشت از لاگ و بکاپ و صفحه |
| آرکیتکت | مدل‌سازی تهدید، SDKها و شعاع انفجار مجوزها، پلتفرم امن به‌صورت پیش‌فرض |



**اسماعیل مرادی**
@esmaeeil-moradi700

# مدل ذهنی: sandbox در دارد




- sandbox **فایل‌های** شما را از اپ‌های دیگر محافظت می‌کند — نه **درهای ورودی** را
- هر چیزی که از یک در عبور می‌کند **ورودی نامطمئن** است: extraها، دیپ‌لینک‌ها، صفحه‌های وب، callbackهای SDK
- فرض کنید **صاحب دستگاه** هم می‌تواند مهاجم باشد: از کلیدها محافظت کنید و تصمیم نهایی را روی سرور بگیرید

**از دارایی‌ها شروع کنید، نه از کتابخانه‌ها**

«چه چیزی ارزش دزدیدن دارد، چه کسی آن را می‌خواهد، و از کدام در وارد می‌شود؟» — قبل از انتخاب هر API رمزنگاری، به این سؤال جواب بدهید.



**اسماعیل مرادی**
@esmaeeil-moradi700

# جونیور

مدل امنیتی پلتفرم، داده کجا ذخیره می‌شود، مجوزهای runtime



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱. مدل امنیتی اندروید در یک جدول

| لایه | چه چیزی به شما می‌دهد |
| --- | --- |
| **App sandbox** — یک UID لینوکسی برای هر اپ، با اعمال SELinux | اپ‌های دیگر نمی‌توانند فایل‌های خصوصی یا حافظهٔ شما را بخوانند |
| **امضای اپ** | آپدیت باید با همان کلید (یا زنجیرهٔ چرخش آن) امضا شده باشد |
| **مجوزها** | دسترسی به داده و APIهای حساس به اجازهٔ صریح نیاز دارد |
| **Verified Boot و رمزنگاری فایل‌محور** | سیستم‌عامل دست‌نخورده؛ دادهٔ ذخیره‌شده رمز شده و به قفل صفحه گره خورده |
| **Android Keystore** — روی TEE یا StrongBox | کلیدی که هیچ‌وقت وارد پروسس شما نمی‌شود |

رمزنگاری فایل‌محور (file-based encryption) روی دستگاه‌هایی که با اندروید ۱۰ یا جدیدتر عرضه می‌شوند اجباری است. **اصل کمترین دسترسی** همه را به هم وصل می‌کند: فقط چیزی را بخواهید که فیچر لازم دارد.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱. چیزهایی که sandbox پوشش نمی‌دهد

**تلهٔ مصاحبه**

«*sandbox اپ من را امن می‌کند.*» — sandbox فقط **فایل‌ها و حافظه** را جدا می‌کند. برای یک Activity اکسپورت‌شده که به extraهایش اعتماد می‌کند، یک WebView که محتوای مهاجم را باز می‌کند، توکنی که در logcat چاپ شده، یا کاربری با دسترسی root که پوشهٔ دادهٔ شما را باز می‌کند، هیچ کاری نمی‌کند.

**تلهٔ مصاحبه**

«*حافظهٔ داخلی رمز شده، پس رازها آنجا امن‌اند.*» — رمزنگاری فایل‌محور از دستگاه **خاموش یا قفل** محافظت می‌کند. وقتی اپ شما در حال اجراست، فایل‌ها برای root قابل‌خواندن‌اند و به‌صورت پیش‌فرض در **Auto Backup** هم می‌روند (سؤال ۲).

**سؤال تکمیلی**

«*از کجا می‌فهمید از چه چیزی باید محافظت کنید؟*» — با مدل‌سازی تهدید: دارایی‌ها، مهاجم‌ها، درهای ورودی، راهکارها. این همان سؤال ۱۱ است.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۲. داده کجا ذخیره می‌شود — و چه کسی می‌تواند بخواندش

| محل | چه کسی می‌تواند بخواند | مناسب برای |
|---|---|---|
| حافظهٔ داخلی: `filesDir`، DataStore و Room | فقط اپ شما — و root | دادهٔ اپ؛ رازهای ارزشمند به شکل **رمزشده** |
| حافظهٔ خارجی مخصوص اپ با `getExternalFilesDir` | اپ شما؛ اپ‌های دیگر فقط تا اندروید ۹ | فایل‌های بزرگ و غیرحساس |
| حافظهٔ مشترک، مثل MediaStore و Downloads | کاربر و اپ‌های دیگر | فایل‌هایی که مال کاربرند |
| Android Keystore | هیچ‌کس — کلیدها قابل‌خروج نیستند | **کلید**، نه داده |
| بکاپ ابری و انتقال بین دستگاه‌ها | روی دستگاه دیگری بازیابی می‌شود | فقط چیزی که اجازه می‌دهید |

در اپ‌هایی که اندروید ۷ یا بالاتر را target می‌کنند، `MODE_WORLD_READABLE` یک `SecurityException` پرتاب می‌کند. فایل را با `FileProvider` و یک مجوز موقت روی URI به اشتراک بگذارید — هرگز با مسیر `file://`.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۲. بکاپ: نسخهٔ فراموش‌شدهٔ داده‌های شما

```
<!-- res/xml/data_extraction_rules.xml (Android 12+) -->
<data-extraction-rules>
    <cloud-backup>
        <exclude domain="sharedpref" path="session.xml" />
        <exclude domain="file" path="datastore/" />
    </cloud-backup>
    <device-transfer>
        <exclude domain="sharedpref" path="session.xml" />
        <exclude domain="file" path="datastore/" />
    </device-transfer>
</data-extraction-rules>
```

با `android:dataExtractionRules` به آن ارجاع بدهید؛ برای اندروید ۱۱ و پایین‌تر هم `android:fullBackupContent` را نگه دارید.

**تلهٔ مصاحبه**

«با `allowBackup="false"` *بکاپ خاموش می‌شود.*» — در اپ‌هایی که اندروید ۱۲ یا بالاتر را target می‌کنند، این فقط بکاپ **ابری** را خاموش می‌کند، **نه انتقال مستقیم بین دو دستگاه** را. فایل‌های حساس را صریحاً در بخش `<device-transfer>` استثنا کنید.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۳. مجوزهای runtime به روش امروزی

```
@Composable
fun ScanButton(onScan: () -> Unit) {
    val context = LocalContext.current
    val launcher = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { granted -> if (granted) onScan() else showWhyNeeded() }

    Button(onClick = {
        val granted = ContextCompat.checkSelfPermission(
            context, Manifest.permission.CAMERA,
        ) == PackageManager.PERMISSION_GRANTED
        if (granted) onScan()
        else launcher.launch(Manifest.permission.CAMERA)
    }) { Text("Scan receipt") }
}
```

- **در موقعیت درست** بپرسید، همان لحظه‌ای که کاربر از فیچر استفاده می‌کند — نه همه را موقع اجرای اپ
- `shouldShowRequestPermissionRationale()` می‌گوید کِی اول باید توضیح بدهید



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۳. انواع مجوز و قواعدشان

| نوع | چطور داده می‌شود | مثال |
|---|---|---|
| زمان نصب (normal) | خودکار | `INTERNET`، `VIBRATE` |
| زمان اجرا (dangerous) | دیالوگ سیستم، با تصمیم کاربر | `CAMERA`، `ACCESS_FINE_LOCATION`، `POST_NOTIFICATIONS` |
| امضا (signature) | فقط اپ‌هایی که با همان کلید امضا شده‌اند | مجوزهای سفارشی خودتان |
| ویژه (app op) | کلید روشن/خاموش در تنظیمات | `SYSTEM_ALERT_WINDOW`، `SCHEDULE_EXACT_ALARM` |

- از اندروید ۱۱: مجوز **یک‌باره** برای موقعیت مکانی، دوربین و میکروفون؛ و **ریست خودکار** مجوزهای اپ‌هایی که مدتی استفاده نشده‌اند

**تلهٔ مصاحبه**

«*اگر کاربر رد کرد، اجرای بعدی دوباره می‌پرسیم.*» — از اندروید ۱۱، اگر کاربر یک مجوز را **دو بار** رد کند، سیستم دیگر دیالوگ را نشان نمی‌دهد و `launch()` فوراً `false` برمی‌گرداند. ارزش فیچر را توضیح بدهید، بعد کاربر را به تنظیمات اپ ببرید.

**تلهٔ مصاحبه**

«*اپ چراغ‌قوه به `CAMERA` نیاز دارد.*» — `CameraManager.setTorchMode()` هیچ مجوزی لازم ندارد. درخواست `CAMERA` در اینجا دقیقاً همان زیاده‌خواهی‌ای است که اصل کمترین دسترسی منعش می‌کند.



**اسماعیل مرادی**
@esmaeeil-moradi700

# مید

دسترسی به رسانه و دسترسی جزئی، کامپوننت‌های exported و intentها، PendingIntent، WebView



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۴. عکس و رسانه: انتخاب کنید، نخوانید

```
val pickPhoto = rememberLauncherForActivityResult(
    ActivityResultContracts.PickVisualMedia()
) { uri: Uri? -> uri?.let(viewModel::onPhotoPicked) }

Button(onClick = {
    pickPhoto.launch(
        PickVisualMediaRequest(
            ActivityResultContracts.PickVisualMedia.ImageOnly
        )
    )
}) { Text("Attach lab result") }
```

| نسخه | دسترسی به رسانه |
| --- | --- |
| API 33 | به‌جای `READ_EXTERNAL_STORAGE` سه مجوز آمده: `READ_MEDIA_IMAGES` و `READ_MEDIA_VIDEO` و `READ_MEDIA_AUDIO` |
| API 34 | کاربر می‌تواند **فقط عکس‌های انتخاب‌شده** را بدهد: `READ_MEDIA_VISUAL_USER_SELECTED` |
| همه | **Photo Picker** اصلاً مجوزی لازم ندارد |



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۴. چیزهایی که مصاحبه‌گر چک می‌کند

**تلهٔ مصاحبه**

«`READ_MEDIA_IMAGES` *را می‌گیرم تا کاربر یک عکس پیوست کند.*» — سیاست مجوزهای عکس و ویدیوی گوگل‌پلی این مجوزها را به اپ‌هایی محدود می‌کند که **فیچر اصلی‌شان** به دسترسی گسترده نیاز دارد. بقیه از Photo Picker استفاده می‌کنند.

**تلهٔ مصاحبه**

«*وقتی مجوز داده شد، کل گالری مال من است.*» — در اندروید ۱۴ کاربر ممکن است گزینهٔ **«Select photos»** را بزند. هر بار که صفحه باز می‌شود دسترسی را دوباره چک کنید و راهی برای تغییر انتخاب بگذارید؛ این مجوز وقتی اپ در background است هم ممکن است عوض شود.

**سؤال تکمیلی**

«*URI انتخاب‌شده تا کِی قابل‌خواندن است؟*» — به‌صورت پیش‌فرض تا ری‌استارت دستگاه یا توقف پروسس اپ. فایل را در حافظهٔ اپ کپی کنید، یا اگر واقعاً دسترسی طولانی لازم دارید `takePersistableUriPermission()` را صدا بزنید.

**سؤال تکمیلی**

«*و موقعیت مکانی؟*» — در دو قدم: اول دسترسی foreground (از اندروید ۱۲ کاربر می‌تواند **موقعیت تقریبی** را انتخاب کند)، بعد جداگانه دسترسی background — که از اندروید ۱۱ کاربر آن را در تنظیمات می‌دهد.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۵. کامپوننت‌های exported: هر دری که باز می‌کنید

```
<!-- Only apps signed with our key may call it -->
<permission
    android:name="com.example.permission.SYNC"
    android:protectionLevel="signature" />

<service
    android:name=".sync.SyncService"
    android:exported="true"
    android:permission="com.example.permission.SYNC" />

<!-- Internal screen: unreachable from other apps -->
<activity
    android:name=".internal.DebugActivity"
    android:exported="false" />
```

- در اپ‌هایی که اندروید ۱۲ یا بالاتر را target می‌کنند، هر کامپوننتی که intent filter دارد باید `android:exported` را صریحاً تعریف کند — وگرنه نصب شکست می‌خورد. قبل از آن، داشتن filter **بی‌سروصدا** یعنی `exported="true"` .
- exported یعنی **هر اپی** می‌تواند آن را با یک intent صریح و **هر** **extra**ای اجرا کند.

**تلهٔ مصاحبه**

«*intent filter محدود می‌کند چه کسی صدایش بزند.*» — نه. filterها intentهای ضمنی را مسیریابی می‌کنند؛ کنترل دسترسی نیستند. از `exported="false"` یا یک مجوز `signature` استفاده کنید.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۵. Intent redirection و receiverهای runtime

```
val next = IntentCompat.getParcelableExtra(
    intent, "next", Intent::class.java)

// BAD: launches whatever Intent another app put inside
// startActivity(next)

// GOOD: forward only to our own, known screens
val target = next?.resolveActivity(packageManager)
if (next != null && target != null &&
    target.packageName == packageName &&
    target.className in allowedScreens
) {
    next.removeFlags(
        Intent.FLAG_GRANT_READ_URI_PERMISSION or
            Intent.FLAG_GRANT_WRITE_URI_PERMISSION
    )
    startActivity(next)
}
```

- **Intent redirection** یعنی Activityای اکسپورت‌شده که `Intent` جاسازی‌شده را جلو می‌فرستد، به هر اپی راه می‌دهد تا از طریق شما به کامپوننت‌های **غیر exported** برسد
- برای receiverهای runtime، flag مربوط به `RECEIVER_NOT_EXPORTED` با target اندروید ۱۴ به بالا اجباری است. در دیباگ، `detectUnsafeIntentLaunch()` در StrictMode این الگو را گزارش می‌کند.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۶. PendingIntent: توکنی که *به اسم شما* عمل می‌کند

```
val intent = Intent(context, ReminderReceiver::class.java)
    .setAction(ACTION_TAKE_DOSE)
    .putExtra(EXTRA_REMINDER_ID, reminder.id)

val pending = PendingIntent.getBroadcast(
    context,
    reminder.id.hashCode(), // distinct per reminder
    intent,
    PendingIntent.FLAG_IMMUTABLE or
        PendingIntent.FLAG_UPDATE_CURRENT,
)
```

- سیستم نوتیفیکیشن، `AlarmManager` یا میزبان ویجت بعداً آن را **با هویت و مجوزهای اپ شما** اجرا می‌کنند
- با target اندروید ۱۲ به بالا، یکی از `FLAG_IMMUTABLE` یا `FLAG_MUTABLE` **الزامی** است. پیش‌فرض را تغییرناپذیر بگذارید.
- `FLAG_MUTABLE` فقط وقتی که گیرنده باید داده‌ای را پر کند: پاسخ درون‌خطی (inline reply)، bubbleها، callbackهای geofencing
- همیشه intent **صریح**. با target اندروید ۱۴ به بالا، ساختن `PendingIntent` تغییرپذیر با intent ضمنی exception می‌دهد



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۶. چیزهایی که مصاحبه‌گر چک می‌کند

**تلهٔ مصاحبه**

«*تغییرپذیر و ضمنی اشکالی ندارد — نوتیفیکیشن خودم است.*» — هر کسی که توکن به دستش برسد می‌تواند بخش‌های خالی را پر کند: آن را به کامپوننت **غیر exported** شما بفرستد یا مجوز URI اضافه کند. بعد به اسم اپ شما اجرا می‌شود.

**سؤال تکمیلی**

«*چرا همهٔ نوتیفیکیشن‌ها یک آیتم را باز می‌کنند؟*» — دو `PendingIntent` وقتی request code و action، data، type، component و categoryهای intentشان یکی باشد، **یک توکن‌اند — extraها حساب نمی‌شوند**. برای هر آیتم یک request code جدا بگذارید، یا یک data URI جدا.

**قاعدهٔ سرانگشتی**

به‌صورت پیش‌فرض صریح و تغییرناپذیر. هر `FLAG_MUTABLE` در code review یک خط توجیه لازم دارد.



اسماعیل مرادی
@esmaeeil-moradi700

# سؤال ۷. WebView: تنظیمات و ناوبری

```
webView.settings.apply {
    javaScriptEnabled = true  // only if the page needs it
    allowFileAccess = false   // default when targeting 30+
    allowContentAccess = false
    mixedContentMode = WebSettings.MIXED_CONTENT_NEVER_ALLOW
}
webView.webViewClient = object : WebViewClientCompat() {
    override fun shouldOverrideUrlLoading(
        view: WebView, request: WebResourceRequest,
    ): Boolean {
        val url = request.url
        val trusted = url.scheme == "https" &&
            url.host in allowedHosts
        if (!trusted) openInBrowser(url)
        return !trusted // true = WebView won't load it
    }
}
```

- صفحه‌های محلی: `WebViewAssetLoader` آن‌ها را روی آدرس `https://appassets.androidplatform.net` سرو می‌کند — بدون نیاز به دسترسی `file://`
- صفحه‌های ورود و پرداخت: **Custom Tabs**، نه WebView (روز ۱۲)



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۷. پل JavaScript

```
// Legacy: every page and frame in this WebView can call it
webView.addJavascriptInterface(Bridge(), "Android")

// Better: an allow-list of origins (androidx.webkit)
if (WebViewFeature.isFeatureSupported(
        WebViewFeature.WEB_MESSAGE_LISTENER)
) {
    WebViewCompat.addWebMessageListener(
        webView, "nativeBridge",
        setOf("https://help.example.com"),
    ) { _, message, _, isMainFrame, _ ->
        if (isMainFrame) viewModel.onWebMessage(message.data)
    }
}
```

**تلهٔ مصاحبه**

«`addJavascriptInterface` *از API 17 امن است.*» — API 17 فقط جلوی reflection را گرفت: حالا فقط متدهای `@JavascriptInterface` در دسترس‌اند. اما همین متدها برای **هر** صفحه و iframeای که WebView به آن برسد باز است — یک redirect یا یک تبلیغ کافی است.

**تلهٔ مصاحبه**

صدا زدن `webView.loadUrl(intent.data.toString())` از روی یک دیپ‌لینک — مهاجم انتخاب می‌کند چه چیزی داخل WebView اپ شما اجرا شود، همراه با پل. اول hostها را با allow-list چک کنید.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سینیور

Android Keystore، ذخیره‌سازی بعد از security-crypto، نشت از لاگ و بکاپ و صفحه



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۸. Android Keystore: کلیدی بسازید که هرگز بیرون نمی‌آید

```
fun getOrCreateKey(alias: String): SecretKey {
    val ks = KeyStore.getInstance("AndroidKeyStore")
        .apply { load(null) }
    (ks.getKey(alias, null) as? SecretKey)?.let { return it }

    val spec = KeyGenParameterSpec.Builder(
        alias,
        KeyProperties.PURPOSE_ENCRYPT or
            KeyProperties.PURPOSE_DECRYPT,
    )
        .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
        .setEncryptionPaddings(
            KeyProperties.ENCRYPTION_PADDING_NONE)
        .setKeySize(256)
        .build()
    return KeyGenerator.getInstance(
        KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore",
    ).apply { init(spec) }.generateKey()
}
```

`SecretKey`ی که برمی‌گردد فقط یک **دستگیره** است. رمزنگاری داخل خود Keystore انجام می‌شود — در TEE، یا اگر خواسته باشید در StrongBox.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۸. گزینه‌های مهم کلید

| گزینه | اثر |
|---|---|
| غیرقابل‌خروج (همیشه) | پروسس شما هیچ‌وقت بایت‌های کلید را نمی‌بیند |
| `setIsStrongBoxBacked(true)` از API 28 | یک تراشهٔ امن جدا؛ کندتر؛ اگر دستگاه نداشته باشد **exception می‌دهد** |
| `setUserAuthenticationRequired(true)` | فقط بعد از احراز هویت کاربر قابل استفاده است (روز ۲۰) |
| `setUnlockedDeviceRequired(true)` از API 28 | وقتی دستگاه قفل است قابل استفاده نیست |
| `KeyInfo.securityLevel` از API 31 | کلید واقعاً کجاست: TEE، StrongBox یا نرم‌افزار |

**تلهٔ مصاحبه**

«`setIsStrongBoxBacked(true)` *اگر سخت‌افزار باشد از آن استفاده می‌کند.*» — نسخهٔ پارسال همین را گفته بود و اشتباه است. بدون StrongBox، ساخت کلید `StrongBoxUnavailableException` پرتاب می‌کند؛ آن را بگیرید و به کلید معمولی مبتنی بر TEE برگردید.

**تلهٔ مصاحبه**

*«از کلید داخل Keystore نمی‌شود سوءاستفاده کرد.»* — **بایت‌های** کلید قابل استخراج نیستند، اما کدی که به اسم اپ شما اجرا می‌شود — بدافزار روی دستگاه rootشده یا ابزار hook — همچنان می‌تواند از Keystore بخواهد رمزگشایی کند. برای همین کلیدهای ارزشمند را به احراز هویت کاربر گره می‌زنند.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۹. ذخیره‌سازی امن بعد از security-crypto

کتابخانهٔ Jetpack Security با نام `androidx.security:security-crypto` **منسوخ شده** — `EncryptedSharedPreferences` و `EncryptedFile` هم جزو آن‌اند. جایگزینش یک wrapper دیگر نیست: **یک کلید Keystore، به‌علاوهٔ AES-GCM و همان ذخیره‌سازی‌ای که الان دارید.**

```
class KeystoreCipher(private val alias: String) {
    private fun key(): SecretKey = getOrCreateKey(alias) // Q8

    fun encrypt(plain: ByteArray): ByteArray {
        val c = Cipher.getInstance("AES/GCM/NoPadding")
        c.init(Cipher.ENCRYPT_MODE, key()) // Keystore picks IV
        return c.iv + c.doFinal(plain)     // [IV | ciphertext]
    }

    fun decrypt(blob: ByteArray): ByteArray {
        val c = Cipher.getInstance("AES/GCM/NoPadding")
        val spec = GCMParameterSpec(128, blob, 0, IV_SIZE)
        c.init(Cipher.DECRYPT_MODE, key(), spec)
        return c.doFinal(blob, IV_SIZE, blob.size - IV_SIZE)
    }

    private companion object { const val IV_SIZE = 12 }
}
```



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۹. یک token store روی DataStore

```
class TokenStore(
    private val dataStore: DataStore<Preferences>,
    private val cipher: KeystoreCipher,
) {
    private val key = stringPreferencesKey("refresh_token")

    suspend fun save(token: String) {
        val blob = cipher.encrypt(token.encodeToByteArray())
        val text = Base64.encodeToString(blob, Base64.NO_WRAP)
        dataStore.edit { it[key] = text }
    }

    suspend fun read(): String? =
        dataStore.data.first()[key]
            ?.let { Base64.decode(it, Base64.NO_WRAP) }
            ?.let { cipher.decrypt(it).decodeToString() }
}
```

**تلهٔ مصاحبه**

«`EncryptedSharedPreferences` *روش توصیه‌شده است.*» — بود؛ نسخهٔ پارسال هنوز آن را «مدرن و توصیهٔ رسمی» می‌نامید. گوگل کل کتابخانه را منسوخ کرد و به APIهای پلتفرم و استفادهٔ مستقیم از Keystore ارجاع می‌دهد. به چند کلید یا چرخش کلید نیاز دارید؟ از Tink با یک keyset که با کلید Keystore رمز شده استفاده کنید.

**تلهٔ مصاحبه**

کلیدهای Keystore **هیچ‌وقت بکاپ نمی‌شوند**. اگر متن رمزشده روی گوشی جدید بازیابی شود، رمزگشایی شکست می‌خورد. آن را از بکاپ استثنا کنید (سؤال ۲) و `AEADBadTagException` را به معنای «پاک کن و دوباره وارد شو» بگیرید، نه کرش.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۰. رازها کجا بدون هیچ «هکی» نشت می‌کنند

| کانال | نشت رایج | راه‌حل |
|---|---|---|
| Logcat | توکن و دادهٔ شخصی در `Log.d` | حذف لاگ‌ها در release؛ logger بی‌اثر |
| SDKهای کرش و آنالیتیکس | ایمیل و بدنهٔ درخواست در breadcrumbها | پاک‌سازی قبل از ارسال؛ allow-list فیلدها |
| لاگ HTTP | هدر `Authorization` در لاگ‌های `BODY` | در release مقدار `Level.NONE`؛ `redactHeader()` |
| بکاپ | refresh token روی دستگاه دیگری بازیابی می‌شود | `dataExtractionRules` (سؤال ۲) |
| کلیپ‌بورد | OTP کپی‌شده در پیش‌نمایش کلیپ‌بورد دیده می‌شود | علامت حساس بزنید (API 33)؛ پاکش کنید |
| نوتیفیکیشن | جواب آزمایش روی صفحهٔ قفل | `VISIBILITY_PRIVATE` همراه با یک نسخهٔ عمومی |
| صفحه و Recents | OTP در اسکرین‌شات | `FLAG_SECURE` (روز ۲۰) |

توصیهٔ پارسال برای صفر کردن رمزهای عبور در `CharArray` هنوز کمی کمک می‌کند — اما runtime ممکن است قبلاً از آن‌ها کپی گرفته باشد. برد بزرگ‌تر، بستن کانال‌های بالاست.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۰. لاگ گرفتن امن

```
# proguard-rules.pro — drop verbose logs in release
-assumenosideeffects class android.util.Log {
    public static int v(...);
    public static int d(...);
}
```

```
val logging = HttpLoggingInterceptor().apply {
    level = if (BuildConfig.DEBUG) Level.BODY else Level.NONE
    redactHeader("Authorization")
    redactHeader("Cookie")
}
```

**تلهٔ مصاحبه**

«*در release کسی* `Log.d` *را نمی‌بیند.*» — هر کسی با USB debugging می‌تواند `adb logcat` بگیرد، و لاگ‌ها در bug reportهایی که مردم به تیکت‌ها پیوست می‌کنند سر درمی‌آورند. اگر راز است، لاگش نکنید — در هیچ build typeی.

**یک‌جا بکارید، همه‌جا نپاشید**

همهٔ لاگ‌ها را از یک logger عبور بدهید (مثلاً Timber با درختی که فقط در دیباگ کاشته می‌شود)، تا «هیچ رازی در لاگ نباشد» یک تصمیم باشد، نه هزار نقطهٔ فراخوانی.



**اسماعیل مرادی**
@esmaeeil-moradi700

# آرکیتکت

مدل‌سازی تهدید، SDKها و شعاع انفجار مجوزها، پلتفرم امن به‌صورت پیش‌فرض



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۱. مدل‌سازی تهدید یک فیچر در یک ساعت




۱. **جریان داده را بکشید** و هر مرز اعتماد (trust boundary) را علامت بزنید

۲. **دارایی‌ها را فهرست کنید:** توکن‌ها، دادهٔ سلامت، جابه‌جایی پول

۳. **در هر عبور از مرز، STRIDE را مرور کنید** (اسلاید بعد)

۴. **تصمیم بگیرید:** رفع، پذیرش، یا انتقال منطق به سرور — و برایش تیکت بسازید

**تلهٔ مصاحبه**

*«اپ سقف را اعمال می‌کند، پس سرور می‌تواند به آن اعتماد کند.»* — هر چیزی داخل APK قابل patch است. قیمت‌ها، سقف‌ها، نقش‌ها و دسترسی به فیچرها **روی سرور** اعتبارسنجی می‌شوند؛ کلاینت فقط برای UX آن‌ها را نشان می‌دهد.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۱. STRIDE به زبان اندروید

| تهدید | مثال اندرویدی | راهکار |
|---|---|---|
| **S**poofing — جعل هویت | یک اپ جعلی سرویس exported شما را صدا می‌زند | مجوز `signature`؛ چک کردن صدازننده |
| **T**ampering — دست‌کاری | APK بسته‌بندی‌شدهٔ مجدد، پروسس hookشده | چک یکپارچگی سمت سرور (روز ۲۰) |
| **R**epudiation — انکار | «من آن انتقال را نفرستادم» | لاگ ممیزی سرور؛ درخواست امضاشده (روز ۲۰) |
| **I**nformation disclosure — افشای اطلاعات | توکن در لاگ یا بکاپ | سؤال ۹ و ۱۰ |
| **D**enial of service — از کار انداختن | کرش با extraهای خراب | اعتبارسنجی همهٔ ورودی‌های intent |
| **E**levation of privilege — ارتقای دسترسی | Intent redirection، `PendingIntent` تغییرپذیر | سؤال ۵ و ۶ |

**سؤال تکمیلی**

«*مالک مدل تهدید کیست؟*» — تیم فیچر، با بازبینی یک security champion. کنار design doc زندگی می‌کند و وقتی جریان داده عوض شد به‌روز می‌شود — نه اینکه یک بار برای ممیزی نوشته شود.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۲. SDKها و شعاع انفجار مجوزها

هر SDK **داخل پروسس شما، با UID شما و با همهٔ مجوزهای شما** اجرا می‌شود، و manifestش **با manifest شما ادغام می‌شود.**

```
<manifest xmlns:tools="http://schemas.android.com/tools">
    <!-- An SDK asked for it; no feature of ours needs it -->
    <uses-permission
        android:name="android.permission.READ_PHONE_STATE"
        tools:node="remove" />
</manifest>
```

```
# CI: fail when the release APK's permission list changes
apkanalyzer manifest permissions app-release.apk > perms.txt
diff perms.baseline.txt perms.txt
```

- مجوز `QUERY_ALL_PACKAGES` با سیاست پلی محدود شده — به‌جایش `<queries>` محدود تعریف کنید
- Google Play SDK Index نسخه‌هایی از SDKها را که مشکل سیاستی یا امنیتی شناخته‌شده دارند علامت می‌زند



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۲. چیزهایی که مصاحبه‌گر چک می‌کند

**تلهٔ مصاحبه**

«*اینکه SDK آنالیتیکس چه جمع می‌کند مشکل فروشنده است.*» — نه در گوگل‌پلی: فرم **Data safety** شما باید داده‌هایی را که SDKهایتان جمع می‌کنند اعلام کند. نشت آن‌ها، تخلف سیاستی شماست.

**تلهٔ مصاحبه**

«*ما فقط یک کتابخانه اضافه کردیم، نه یک مجوز.*» — ادغام manifest آن را برایتان اضافه کرد. **manifest ادغام‌شده** و APK نهایی را بررسی کنید، نه فقط `AndroidManifest.xml` خودتان را.

**سؤال تکمیلی**

«*یک SDK جدید را چطور ارزیابی می‌کنید؟*» — مجوزها و کامپوننت‌هایی که اضافه می‌کند؛ چه چیزی روی شبکه می‌فرستد (ترافیک را بررسی کنید)؛ اینکه آیا راه‌اندازی‌اش می‌تواند تا رضایت کاربر صبر کند؛ سابقهٔ نگه‌داری و CVEها؛ وضعیتش در SDK Index.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۳. امن به‌صورت پیش‌فرض: مسیر امن را مسیر آسان کنید

```
// build-logic convention plugin, applied to every app module
extensions.configure<ApplicationExtension> {
    lint {
        error += setOf(
            "SetJavaScriptEnabled",
            "AddJavascriptInterface",
            "ExportedService",
            "UnspecifiedImmutableFlag",
            "TrustAllX509TrustManager",
            "HardcodedDebugMode",
        )
        abortOnError = true
        checkReleaseBuilds = true
    }
}
```

Lint روی هر pull request اجرا می‌شود، پس یک پیش‌فرض ناامن را build fail می‌کند، به‌جای اینکه منتظر تست نفوذ بماند.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۳. جادهٔ آسفالته (paved road)

| به‌صورت مرکزی فراهم کنید | تا تیم‌ها خودشان دست‌ساز ننویسند |
|---|---|
| ماژول `:core:security` شامل `KeystoreCipher` و `TokenStore` | رمزنگاری با mode اشتباه یا IV ثابت |
| یک factory برای `SecureWebView` با originهای مجاز | تنظیمات WebView جداگانه برای هر صفحه |
| helperهایی که intent صریح و تغییرناپذیر می‌سازند | `PendingIntent`های تغییرپذیر و ضمنی |
| قواعد Lint در CI، و StrictMode در دیباگ | اعمال قواعد فقط با code review |
| قالب مدل تهدید داخل design doc | یافته‌های امنیتی یک هفته قبل از انتشار |

**نظر من**

تیم‌های امنیت با بازبینی تک‌تک PRها مقیاس‌پذیر نمی‌شوند. با ساختن یک جادهٔ آسفالته مقیاس‌پذیر می‌شوند — و اجازه می‌دهند build به مسیر آسفالت‌نشده «نه» بگوید.

**تلهٔ مصاحبه**

«*امنیت یعنی یک تست نفوذ قبل از انتشار بزرگ.*» — یافته‌هایی که این‌قدر دیر پیدا شوند گران‌اند و آخرش «پذیرفته» می‌شوند. مدل تهدید در زمان طراحی، Lint در زمان PR، و تست نفوذ برای **تأیید**.



**اسماعیل مرادی**
@esmaeeil-moradi700

# روز ۱۹ — خلاصهٔ یک‌صفحه‌ای

- sandbox فایل‌ها را جدا می‌کند، نه درها را: کامپوننت exported، intent، WebView، لاگ و بکاپ همه در هستند.
- کمترین دسترسی: مجوز را در موقعیت درست بخواهید؛ Photo Picker **هیچ** مجوزی نمی‌خواهد.
- بعد از دو بار رد (اندروید ۱۱ به بعد) دیالوگ دیگر نمی‌آید — توضیح بدهید و به تنظیمات ببرید.
- `exported` را صریح بنویسید؛ filter کنترل دسترسی نیست؛ `Intent` جاسازی‌شده را کورکورانه جلو نفرستید.
- در `PendingIntent` صریح و `FLAG_IMMUTABLE`؛ extraها توکن‌ها را متمایز نمی‌کنند.
- در WebView فقط در صورت نیاز JavaScript، بدون دسترسی فایل، hostهای مجاز و `addWebMessageListener`.
- `security-crypto` منسوخ است: کلید Keystore با AES-GCM روی DataStore؛ متن رمزشده را از بکاپ استثنا کنید.
- StrongBox اگر نباشد exception می‌دهد؛ Keystore از بایت‌های کلید محافظت می‌کند، نه از استفاده از آن.
- مدل تهدید برای هر فیچر، ممیزی مجوز SDKها، و Lint در CI.

**روز ۲۰ در راه است: امنیت، بخش ۲ — یکپارچگی، بیومتریک و passkey.** این پست را ذخیره کنید و همراه ما باشید.



**اسماعیل مرادی**
@esmaeeil-moradi700